LangGraph v1.0.x

Section 03: Conditional Routing & Branching

Agentic AI Foundations

March 2026

Making Graphs Dynamic with Conditional Logic

Conditional Routing

Simple graphs are linear. Agents need to make *decisions*. Conditional edges route based on state.

Examples

- Tool use agent: "Did LLM call a tool? Route to tool executor."
- Classifier: "Route to approve or reject path."
- Retry loop: "Did it fail? Go back to try again."

Why Conditional Routing: Decision-Making

Linear Isn't Enough

Linear graphs execute the same sequence every time. Agents need to *decide* where to go next based on state.

Conditional Edges Enable

- Branching paths (A -¿ B or C or D)
- Looping (A -¿ A again)
- Early exit (-¿ END)
- Dynamic workflows

Router Functions: Read State, Return Next Node

Router Signature

A router function reads state and returns a *string* naming the next node.

Simple Router

```
def my_router(state: State) -> str:
    if state["score"] > 0.5:
        return "approve"
    else:
        return "reject"
```

Router returns a string. The graph looks it up in a path map.

add_conditional_edges: Source, Router, Path Map

Conditional Edges

```
graph.add_conditional_edges(
    "source_node",          # From
    my_router,              # Router function
    {
        "approve": "approve_node",
        "reject": "reject_node"
    }
)
```

Three Parts

- Source node (where we're coming from)
- Router function (logic to decide)
- Path map (string -¿ node name)

Multiple Conditional Branches

Router with Many Paths

```python
def route_action(state: State) -> str:
    action = state["action"]
    if action == "approve":
        return "approve"
    elif action == "reject":
        return "reject"
    else:
        return "review"

graph.add_conditional_edges(
    "classifier",
    route_action,
    {
        "approve": "approve_node",
        "reject": "reject_node",
        "review": "review_node"
    }
)
```

Default/Fallback Routing

When Router Returns Unknown Value

Set a default fallback path.

With Default

```
graph.add_conditional_edges(
    "router",
    router_fn,
    {
        "path_a": "node_a",
        "path_b": "node_b"
    },
    default="node_default"  # Fallback
)
```

If router returns something not in the map, default catches it.

Cycles and Loops: Agentic Feedback

Loop Pattern

A node can route back to itself (or an earlier node) for iteration.

Use Cases

- Agent think-¿act→observe→think cycle
- Retry logic (failed? try again)
- Refinement loops

LangGraph handles cycles natively. No special syntax needed.

Breaking Out of Loops: Route to END

Termination Condition

A loop must have a condition that routes to END, or it runs forever!

Loop with Exit

```python
def should_continue(state: State) -> str:
    if state["done"]:
        return "done"  # Route to END
    else:
        return "loop"  # Route back

graph.add_conditional_edges(
    "agent",
    should_continue,
    {"done": END, "loop": "agent"}
)
```

tools_condition: Routing on Tool Use

Common Pattern

Did the LLM call tools? Route to tool executor, else to END.

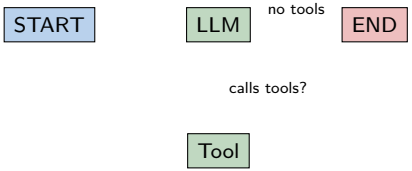
Using tools_condition

```
from langgraph.prebuilt import tools_condition

graph.add_conditional_edges(
    "agent",
    tools_condition,
    {
        "tools": "execute_tools",
        "end": END
    }
)
```

`tools_condition` checks if last message has tool_calls.

Complete Agent Loop: Model -¿ Tools → Model → END



The agent loop: think (LLM) -¿ decide (router) → act (tools) → think again.

Branching Patterns: Fan-Out and Fan-In

Fan-Out

One node routes to multiple nodes in parallel (or sequence).

Fan-In

Multiple nodes converge to one. All must complete before proceeding.

Use Cases

- Fan-out: process multiple paths
- Fan-in: aggregate results

Map-Reduce: Process Items, Then Aggregate

Pattern

Fan-out to process each item, then fan-in to aggregate results.

Idea

```
# State: items: [1, 2, 3, ...]
# Map: process each
# Reduce: combine results
```

LangGraph supports this with conditional edges + loops.

Send API for Dynamic Fan-Out

Dynamic Routing

For *unknown* number of paths, use Send() to create edges dynamically.

Send Example

```
from langgraph.graph import Send

def route_to_many(state):
    return [
        Send("worker", {"item": x})
        for x in state["items"]
    ]
```

Advanced pattern. Useful for variable-width fan-out.

Common Mistakes: Infinite Loops

Mistake 1: Infinite Loop

```
# WRONG: routes back to self forever
graph.add_conditional_edges(
    "agent",
    my_router,
    {"continue": "agent"}
    # Missing exit condition
)
```

Fix

Always have a condition that routes to END!

Common Mistakes: Missing Paths in Map

Mistake 2: Router Returns Unregistered Path

```
# Router returns "unknown"
def router(state): return "unknown"

# But path map doesn't have it
graph.add_conditional_edges(
    "node",
    router,
    {"path_a": "a", "path_b": "b"}
    # "unknown" not mapped!
)
```

Fix

Use default, or ensure router only returns known paths.

Advanced Patterns Summary

Sequential Branching

Node A -¿ (B or C or D) in sequence.

Parallel Branching

Node A -¿ multiple nodes at once (with Send).

Looping

Node A -¿ (A again or continue).

Hierarchical

Subgraphs within graphs (advanced).

Summary: Conditional Routing & Branching

Core Concepts

- Router functions: read state, return node name
- add_conditional_edges: source, router, path map
- Cycles: loops for agent reasoning
- Fan-out/fan-in: branching and aggregation
- Send API: dynamic routing

Critical

Always have an exit condition for loops!

Pattern

Agent loop = LLM -¿ router → tool → loop back (or END)

Self-Check Questions

1. What does a router function return?
2. How do conditional edges work (3 parts)?
3. What's the difference between fan-out and fan-in?
4. Why do loops need an exit condition?
5. When would you use tools_condition?
6. What does Send API do?
7. How do you implement an agent loop?

Quick Answers: (1) String (node name). (2) Source, router fn, path map. (3) Fan-out: one-¿many; fan-in: many→one. (4) Without exit, infinite loop. (5) Route on LLM tool calls. (6) Dynamic fan-out. (7) LLM→router→tool→loop/END.

Next Steps Beyond Section 03

You Now Know

- State, nodes, edges (Section 01)
- Building graphs (Section 02)
- Conditional routing & branching (Section 03)

Future Topics

Persistence, tools, streaming, human-in-the-loop, deployment, evaluation.

You're ready to build production agentic workflows!